Code Style and Engineering Standards

This document defines the mandatory coding standards for the entire codebase. These rules apply to application code, tests, scripts, configuration, migrations, API contracts, logs, documentation embedded in code, commit-ready fixtures, and generated examples maintained by the team.

1. Language

- Write all source code in English.
- Use English for file names, directory names, identifiers, types, interfaces, classes, functions, variables, database objects, API fields, error codes, logs, tests, and technical comments.
- Use English for internal developer documentation and OpenAPI operation identifiers.
- User-facing text may be localized according to product requirements, but it must not be used as an internal identifier.
- Do not mix English and Spanish in names such as `createOrden`, `tecnicoService`, or `clienteId`.
- Prefer clear domain terms over literal or ambiguous translations.

```
// Correct
const assignedTechnicianId = order.technicianId;

// Incorrect
const tecnicoAsignadoId = orden.technicianId;
```

2. Production-ready baseline

Every committed change must be suitable for production unless it is explicitly isolated as a prototype.

- Do not commit placeholder implementations, unexplained hardcoded values, debug statements, dead code, or commented-out code.
- Do not use `TODO`, `FIXME`, or `HACK` without a tracked issue, a reason, and a clear removal condition.
- Handle expected failure modes explicitly.
- Validate all external input and configuration at system boundaries.
- Use secure defaults and least-privilege access.
- Keep secrets, credentials, tokens, and private URLs outside source control and logs.
- Make operations that can be retried idempotent when duplication could cause harm.
- Preserve backward compatibility for public contracts unless a breaking change is intentional and documented.
- Include tests proportional to the risk and behavior introduced by the change.

- Update technical documentation when behavior, configuration, or architecture changes.

3. Formatting and automated checks

- All files must be formatted before they are committed.
- Use the repository formatter and linter as the source of truth; do not maintain personal formatting conventions.
- Do not disable formatter or linter rules without a documented technical reason.
- Keep exceptions as narrow as possible and place them next to the affected code.
- Run formatting, linting, type checking, and relevant tests before considering work complete.
- CI must reject unformatted code, lint errors, type errors, failing tests, and invalid OpenAPI documents.
- Use UTF-8, Unix line endings, and a final newline in text files.
- Remove trailing whitespace.
- Let automated tools decide whitespace, indentation, quotes, semicolons, and line wrapping.

Expected validation commands:

```
pnpm format
pnpm lint
pnpm typecheck
pnpm test
pnpm build
```

4. Naming

- Choose names that describe intent, domain meaning, and units when relevant.
- Avoid vague names such as `data`, `item`, `value`, `object`, `thing`, `manager`, `helper`, or `utils` when a more precise term exists.
- Do not use unexplained abbreviations.
- Use `camelCase` for variables, functions, object properties, and module instances.
- Use `PascalCase` for types, interfaces, classes, Vue components, and enum-like schemas.
- Use `UPPER_SNAKE_CASE` only for true constants and environment variable names.
- Use `kebab-case` for general file and directory names unless a framework convention requires otherwise.
- Name Vue component files with `PascalCase.vue`.
- Name tests after the unit or behavior under test.
- Use boolean names that read as predicates, such as `isActive`, `hasEvidence`, `canComplete`, or `shouldRetry`.
- Include units in names when ambiguity is possible, such as `timeoutMs`, `sizeBytes`, or `ttlSeconds`.
- Use plural nouns for collections and singular nouns for individual values.

5. Code structure

- Organize the codebase by business domain using vertical slices.
- Keep module boundaries explicit and expose a small public API.
- Do not import private internals from another module through deep paths.
- Keep controllers thin: translate HTTP input into one application call and translate its result into HTTP output.
- Keep business rules in domain or application services, not in controllers, routes, Vue views, or persistence adapters.
- Keep infrastructure integrations behind interfaces or ports.
- Keep Prisma models internal to the API; use shared contracts for communication with the frontend.
- Keep remote server state in TanStack Query and reserve Pinia for session and truly global UI state.
- Extract shared code only after real reuse or when it represents a global architectural rule.
- Prefer focused modules with high cohesion over broad `common`, `helpers`, or `utils` collections.
- Avoid circular dependencies.
- Keep dependency direction toward domain and application logic.

6. Functions and control flow

- Write small, focused functions with one clear responsibility.
- Prefer early returns and guard clauses over deeply nested conditionals.
- Keep side effects visible and localized.
- Separate validation, decision-making, persistence, and external I/O when doing so clarifies behavior.
- Avoid boolean parameters when they make a call ambiguous; prefer an options object or separate functions.
- Do not mutate inputs unless the API explicitly communicates mutation.
- Prefer immutable transformations for application data.
- Avoid premature abstractions and generic frameworks built for a single use case.
- Avoid clever code that saves lines at the cost of readability or debuggability.
- Do not catch errors unless the code can add context, translate them, recover safely, or perform required cleanup.
- Never leave an empty `catch` block.

7. TypeScript

- Keep TypeScript strict mode enabled.
- Do not use `any`. Use `unknown` at untrusted boundaries and narrow it through validation or type guards.

- Do not use unsafe type assertions to bypass compiler errors.
- Prefer inferred local types and explicit types at public boundaries.
- Model meaningful domain states with discriminated unions, enums, or validated schemas.
- Avoid optional fields when absence is not a valid domain state.
- Use `readonly` where mutation is not expected.
- Derive DTO types from Valibot schemas instead of defining parallel types manually.
- Do not expose Prisma-generated types as API contracts.
- Handle nullable and optional values deliberately.
- Use exhaustive checks for finite state transitions and discriminated unions.

8. Validation and API contracts

- Treat every external value as `unknown` until validated.
- Validate request parameters, query strings, bodies, environment variables, Sheets data, storage metadata, and external API responses.
- Define reusable request and response contracts with Valibot.
- Keep JSON fields in `camelCase` and timestamps in ISO 8601 UTC.
- Use stable, machine-readable error codes and safe user-facing messages.
- Never expose stack traces, internal exception text, SQL details, credentials, or provider secrets to clients.
- Declare allowed filters and sortable fields explicitly; never map arbitrary query parameters to Prisma.
- Keep OpenAPI documentation synchronized with runtime validation and actual responses.
- Make breaking API changes explicit and versioned.

9. Error handling and debuggability

Code must be easy to diagnose without exposing sensitive information.

- Use typed domain and application errors for expected failures.
- Add actionable context when translating an error between layers.
- Preserve the original error as a cause when the runtime supports it.
- Assign and propagate a `requestId` for every HTTP request.
- Include relevant identifiers in structured logs, such as `requestId`, `userId`, `orderId`, `module`, `action`, duration, and outcome.
- Log events at the appropriate level: `debug`, `info`, `warn`, or `error`.
- Do not use `console.log` in application code; use the shared logger.
- Never log passwords, JWTs, cookies, authorization headers, private keys, signed URLs, binary content, or unnecessary personal data.
- Avoid generic messages such as `Something failed` when a safe and specific error code can identify the operation.
- Do not use logs as a substitute for correct error propagation or monitoring.

- Ensure background jobs report attempts, result, duration, retry state, and sanitized failure reason.

10. Comments and documentation in code

- Write comments only when they provide necessary technical context that the code cannot express clearly.
- Comments must explain why a constraint, workaround, invariant, or non-obvious decision exists.
- Do not write vague, decorative, conversational, or redundant comments.
- Do not narrate what the next line already says.
- Do not use comments to compensate for poor naming or excessive complexity; improve the code first.
- Keep comments accurate when behavior changes.
- Reference an issue, external limitation, or specification when a workaround depends on it.
- Use JSDoc only for public APIs or behavior that cannot be understood from the signature and types.
- Do not add file header comments that repeat the file name or module name.

```
// Correct: the row can move when users sort the spreadsheet, so it must be
// resolved again from its external ID and IMEI immediately before writing.
const currentRow = await equipmentSheet.findCurrentRow(equipmentIdentity);

// Incorrect: get the current row
const currentRow = await equipmentSheet.findCurrentRow(equipmentIdentity);
```

11. Security

- Apply authentication and authorization in the backend, regardless of frontend visibility.
- Check both role capability and resource ownership where required.
- Use parameterized database queries.
- Restrict CORS to an explicit allowlist when credentials are enabled.
- Set secure cookie attributes according to the deployment environment.
- Validate JWT signature, issuer, audience, purpose, and expiration.
- Hash passwords with Argon2id and store only refresh-token hashes.
- Give Google Sheets and S3 credentials the minimum required permissions.
- Keep object storage private and use short-lived signed URLs.
- Validate uploaded object existence, type, and size before marking it complete.
- Protect against user enumeration and excessive authentication attempts.
- Review dependencies and container images for known vulnerabilities in CI.

12. Database and persistence

- Use Prisma migrations for schema changes.
- Give every migration a clear, reviewable purpose.
- Do not edit a migration that has already been applied to a shared environment.
- Use transactions for local changes that must succeed or fail together.
- Keep external calls outside long-running database transactions.
- Store domain changes and critical outbox operations in the same transaction.
- Enforce important invariants at both application and database levels when possible.
- Add indexes based on actual query patterns and verify their effect.
- Avoid unbounded queries and N+1 access patterns.
- Use optimistic concurrency where concurrent administrative changes can conflict.
- Make destructive migrations reversible or provide a tested backup and rollback procedure.
- Do not store image binaries or Base64 payloads in PostgreSQL.

13. External integrations

- Access Google Sheets, S3, and other providers only through dedicated adapters.
- Set explicit timeouts for every external request.
- Retry only operations classified as safe to retry.
- Use exponential backoff with jitter and a configurable attempt limit.
- Do not retry an ambiguous write until the remote state has been verified.
- Use idempotency keys for assignments, completion, and synchronization operations.
- Use circuit breakers per integration rather than one global breaker.
- Sanitize provider errors before persisting, logging, or returning them.
- Never use a spreadsheet row number as a stable identity.
- Update only explicitly authorized cells in Google Sheets.

14. Vue frontend

- Use Vue 3 Composition API and `<script setup lang="ts">`.
- Keep views focused on rendering and user interaction.
- Place UI orchestration in composables and HTTP details in module API files.
- Keep critical business rules in the backend; frontend validation improves usability but is not authoritative.
- Use TanStack Query for remote data, caching, invalidation, and mutation state.
- Use Pinia only for authenticated session data and truly global UI state.
- Keep persistent filters in the URL.
- Provide explicit loading, empty, error, success, and disabled states.
- Ensure forms show field-level validation and preserve entered data after recoverable failures.

- Build technician workflows mobile-first and verify keyboard and touch interaction.
- Use semantic HTML and accessible labels, focus states, and contrast.
- Do not cache authenticated API responses in the service worker.
- Do not introduce deep imports into another frontend module.

15. Testing

- Test observable behavior, business rules, contracts, and failure modes.
- Use unit tests for pure rules and state transitions.
- Use integration tests for routes, persistence, storage, outbox processing, idempotency, and authorization.
- Use end-to-end tests for critical administrator and technician workflows.
- Include negative tests for permissions, invalid transitions, stale versions, external conflicts, and incomplete orders.
- Keep tests deterministic and independent.
- Do not depend on test execution order.
- Do not use production services or production Google Sheets in automated tests.
- Use factories or builders with meaningful defaults instead of large duplicated fixtures.
- Avoid arbitrary sleeps; wait for observable conditions.
- Name tests as behavior statements.
- A bug fix must include a regression test whenever practical.
- Do not change or weaken a test merely to make an incorrect implementation pass.

16. Dependencies

- Add a dependency only when it provides clear value over a small, maintainable implementation.
- Verify compatibility, maintenance status, license, bundle impact, and security before adoption.
- Use one library for one responsibility when possible; remove redundant packages.
- Pin resolved versions through the lockfile.
- Do not import internal paths from third-party packages unless their public documentation requires it.
- Isolate vendor-specific behavior behind adapters when replacement cost would otherwise spread through the codebase.

17. Change discipline

- Keep each change focused on one coherent outcome.
- Preserve existing behavior unless the change explicitly modifies it.
- Do not include unrelated refactors in a feature or bug fix.
- Remove unused code introduced or made obsolete by the change.

- Update tests, contracts, migrations, environment examples, and documentation together with the implementation.
- Review the final diff for secrets, debug output, generated noise, accidental file changes, and missing error handling.
- Ensure the codebase builds and the relevant validation commands pass before handoff.

18. Prohibited practices

- No emojis in source code, comments, logs, user-facing copy maintained in the repository, documentation, commit-ready examples, or test names.
- No mixed-language identifiers.
- No vague or redundant comments.
- No `any`, silent error swallowing, empty `catch` blocks, or ignored promises.
- No production `console.log` statements.
- No secrets, credentials, tokens, signed URLs, or private keys in the repository.
- No business rules implemented only in the frontend.
- No direct provider SDK usage outside its integration adapter.
- No arbitrary SQL string construction.
- No unbounded list endpoints.
- No committed dead code, commented-out code, temporary debug code, or unexplained magic values.
- No disabling of type checks, lint rules, security controls, or tests merely to make a change pass.

19. Definition of done

A change is complete only when all applicable statements are true:

- [ ] Code and technical identifiers are written in English.
- [ ] The implementation follows module boundaries and has a clear production-ready structure.
- [ ] External input is validated and permissions are enforced in the backend.
- [ ] Expected failures are handled and diagnosable through safe structured logs.
- [ ] Comments are technical, necessary, specific, and current.
- [ ] No emojis, temporary code, secrets, or vague placeholders remain.
- [ ] Formatting, linting, type checking, relevant tests, and build pass.
- [ ] Tests cover new behavior and important failure paths.
- [ ] API contracts, migrations, configuration examples, and documentation are updated when affected.
- [ ] The final diff contains only intentional changes.